

Praktikum 2

CampusManager: Menü und Kontrollstrukturen

Ziele

In diesem Praktikum erweitern Sie den *CampusManager* um eine menügesteuerte Bedienung und mehrere kleine Analysefunktionen.

Sie wenden dabei Kontrollstrukturen wie if/else, switch-case, for, while, do-while sowie robuste Eingabevalidierung an.

Allgemeine Hinweise

- Das Praktikum 2 umfasst 3 Aufgaben, die als Hausaufgabe selbständig im 2er-Team gelöst werden sollen.
- Zum Bestehen des Praktikums müssen alle drei Aufgaben vollständig bearbeitet und funktionsfähig umgesetzt sein.
- Zum Praktikumstermin müssen die erstellten Programme fertig und lauffähig sein und von Ihnen vorgestellt werden.
- Im Git-Repository Ihres Teams finden Sie für dieses Praktikum bereits die drei Klassen `P2_A1_Menu`, `P2_A2_Batch` und `P2_A3_Wochenplan` als Grundgerüst. Nutzen Sie die im Quelltext eingefügten Kommentare als Orientierung und ergänzen Sie dort Ihre eigene Lösung.
- Außerhalb der TH ist zwingend eine VPN-Verbindung zur TH notwendig, um auf Ihr Team-Git-Repository zugreifen zu können. Infos: https://www.th-koeln.de/hochschule/vpn---virtual-private-network_26952.php
- Abgabe:
 - Es ist eine gemeinsame Lösung pro Team abzugeben. Die Abgabe muss bis zum Abend vor dem Vorführtermin, spätestens bis 18:00 Uhr, erfolgen.
 - Abgabeort: Push des veränderten Quellcodes zurück in Ihr Repository. Kontrollieren Sie über die Gitlab-Webseite, dass dies korrekt funktioniert hat.
 - Quellcode: muss kompilierbar und ausführbar sein.
- Vor-Ort-Termin: Die Termine werden weiter aufgesplittet: Ein Termin dauert nur noch 20 Minuten, dafür gibt es mehrere Terminslots an einem Tag.
 - Jedes Team bucht **EINEN** Terminslot (beide Teammitglieder müssen diesen Termin buchen).

Hinweise zur Lösung der Aufgaben

- **Nur Konsolenprogrammierung.**
Es werden **keine Arrays** und **keine zusätzlichen eigenen Klassen** verwendet.
Methoden zur Strukturierung sind erlaubt.
- Alle Eingaben müssen überprüft werden.
Bei ungültigen Eingaben muss die Eingabe wiederholt werden.
- Das Programm soll übersichtlich strukturiert sein und sinnvolle Ausgaben erzeugen.
- Nutzen Sie für alle Eingaben bevorzugt die folgenden einfachen Hilfsmethoden:

```
static String readLine(String prompt) {  
    System.out.print(prompt);  
    return SC.nextLine();  
}  
  
static int readInt(String prompt) {  
    while (true) {  
        try {  
            return Integer.parseInt(readLine(prompt).trim());  
        } catch (Exception e) {  
            System.out.println("Bitte eine ganze Zahl eingeben.");  
        }  
    }  
}  
  
static char readChar(String prompt) {  
    String s = readLine(prompt).trim();  
    return s.isEmpty() ? '\n' : Character.toUpperCase(s.charAt(0));  
}
```

Definieren Sie diese Methoden als Klassenmethoden in Ihren Quelltextdateien
(Definition nach `public class...` und vor `public static void main...`)

Definieren Sie ein Scanner-Objekt nach `public class...` und vor den
Klassenmethoden wie folgt:

```
static final Scanner SC = new Scanner(System.in);
```

Denken Sie auch daran, die Scanner-Klasse am Anfang Ihrer Quelltext-Datei zu
importieren: `import java.util.Scanner;`

Aufgabe 1 – Menü für den CampusManager

Erstellen Sie ein dauerhaft laufendes Menü, das verschiedene Funktionen des CampusManagers anbietet.

Das Menü beendet sich erst, wenn ausdrücklich „Beenden“ gewählt wird.

Das Menü soll wie folgt aussehen:

```
Menü:  
(M) Modul anlegen / bearbeiten  
(A) Modul anzeigen  
(N) Note setzen (1..5)  
(B) Bonusregel prüfen (ECTS % 3 == 0)  
(P) Fortschrittsbalken: ECTS-Ziel  
(T) Zeitraster 09:00-16:55 in 5-Min-Schritten  
(X) Beenden
```

Vorgaben

- Eingabe über ein einzelnes Zeichen (char)
- Groß-/Kleinschreibung wird ignoriert (wird bereits in der vorgegebenen Hilfsfunktion erledigt).
- Steuerung über eine Schleife (while oder do-while) und switch-case.
- Ungültige Eingaben müssen erkannt und mit einer Fehlermeldung beantwortet werden.

Funktionale Hinweise

- **(M) Modul anlegen / bearbeiten**
 - Eingabe von name (String), ects (int > 0), note (1–5).
 - Ungültige Eingaben → Wiederholung.
- **(A) Modul anzeigen**
 - Gibt alle Werte übersichtlich aus.
 - Wenn noch kein Modul angelegt wurde → Hinweis.
- **(N) Note setzen (1–5)**
 - Neue Note eingeben, mit Eingabevalidierung.

-
- **(B) Bonusregel prüfen**
 - Prüft, ob die ECTS-Zahl durch 3 teilbar ist.
 - Ausgabe „Bonus möglich“ oder „kein Bonus“.
 - **(P) Fortschrittsbalken**
 - Ziel-ECTS einlesen (muss > 0 sein).
 - Einen Textbalken anzeigen (z. B. ##### _____). Anzahl der angezeigten Zeichen: Ziel-ECTS
 - Für jeden durch das Modul bereits erreichten ECTS-Punkt wird das Zeichen # ausgegeben, für jeden noch offenen ECTS-Punkt das Zeichen _.
 - Falls die aktuelle ECTS-Zahl größer als das Ziel ist, kann der Balken nur aus # bestehen.
 - **(T) Zeitraster**
 - Ausgabe aller Zeiten von **09:00 bis 16:55** in **5-Minuten-Schritten**.
 - Stunden/Minuten sollen **mit führender 0** ausgegeben werden.
 - Bei **11:15** zusätzlich: „CampusManager: Datenabgleich läuft ...“.
 - **(X) Beenden**
 - Sicherheitsabfrage: „Wirklich beenden? (j/n)“.
 - Programm nur bei Bestätigung beenden.

Aufgabe 2 – Batch-Eingabe von Moduldaten (ohne Arrays)

Schreiben Sie ein Programm, das eine wiederholte Eingabe von Moduldaten ermöglicht – **ohne** Speicherung in Arrays.

Es sollen stattdessen **laufende Kennzahlen** berechnet werden.

Ablauf

1. Eingabe der Anzahl n ($1 \leq n \leq 10$).
2. Für jeden Datensatz ($i = 1 \dots n$):
 - Note eingeben (1–5), danach prüfen.
 - ECTS eingeben (≥ 0), danach prüfen.
 - Zählen Sie:
 - Anzahl bestandener Module (Note ≤ 4)
 - Summe ECTS der bestandenen Module
 - Summe aller ECTS
3. Am Ende Ausgabe eines **Berichts**:
 - Anzahl bestandener Module
 - Summe der ECTS (bestanden / gesamt)
 - Bestandsquote in %
 - Meldung, ob mindestens **30 ECTS bestanden** wurden

Hinweis: Diese Funktion darf optional auch in Aufgabe 1 als zusätzlicher Menüpunkt (R) eingebunden werden.

Aufgabe 3 – Wochenplan-Generator

Erstellen Sie ein Programm, das für mehrere Wochen einen tabellarischen Wochenplan erzeugt.

Jede Woche besteht aus den fünf Tagen Montag bis Freitag.

Vorgaben

Die Ausgabe folgt diesen Regeln:

- **Montag:** -
- **Dienstag:** P (Praktikum) **nur jede dritte Woche** (also Woche 3, 6, 9, ...)
- **Mittwoch:** U (Übung) **nur in geraden Wochen**, sonst -
- **Donnerstag:** V (Vorlesung, immer)
- **Freitag:** P (Praktikum) **nur jede dritte Woche** (also Woche 3, 6, 9, ...)

Eingabe

- Anzahl der Wochen w (mindestens 4)

Beispielausgabe für $w = 7$

W1:	-	-	-	V	-
W2:	-	-	U	V	-
W3:	-	P	-	V	P
W4:	-	-	U	V	-
W5:	-	-	-	V	-
W6:	-	P	U	V	P
W7:	-	-	-	V	-

Tipp: Nutzen Sie geschachtelte Schleifen und if-Bedingungen.